

信息安全基础综合设计实验

Lecture 01

李经纬

电子科技大学

个人介绍

➤ 李经纬，研究员

- 承担**信息安全基础综合设计实验**、**Object-Oriented Technology**授课
- 开展学术研究→做系统，发论文

➤ 联系方式：***jwli@uestc.edu.cn***

课程信息

课程目标

➤前置课程

✓“计算机算法与程序设计”：C/C++语言

✓“信息安全数学基础”：数论

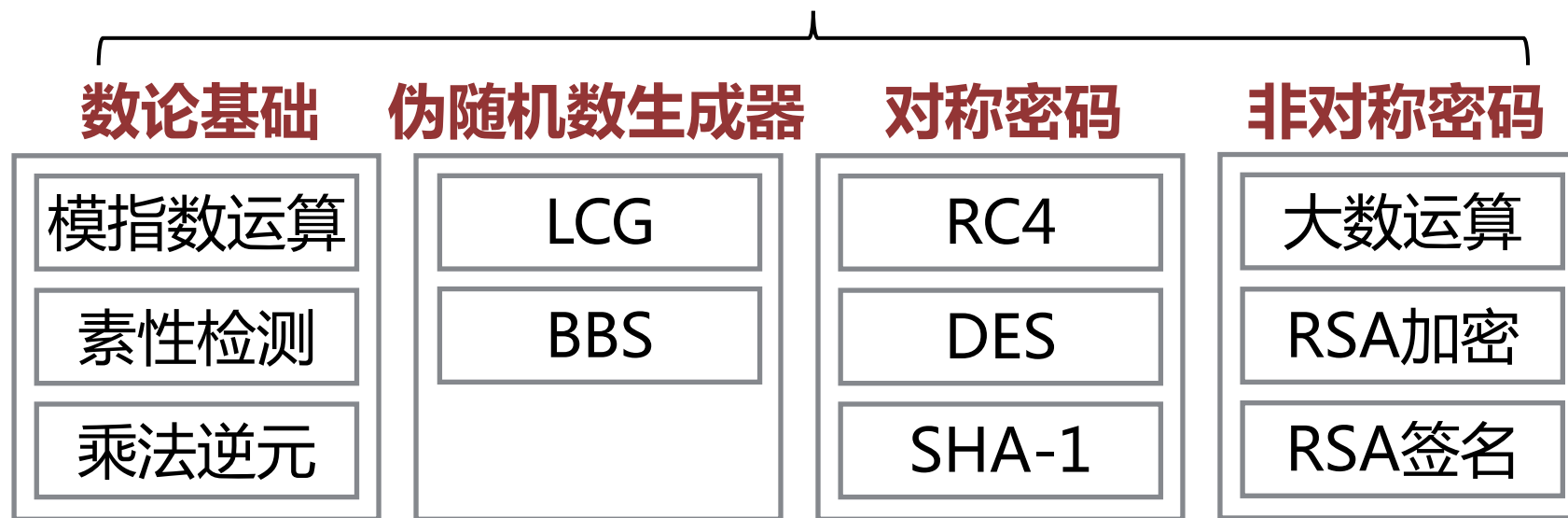
✓“密码学”：加密技术、哈希函数、数字签名

➤目标: 建立综合运用**密码学和数论基础知识**的能力，以及**分析和解决问题的能力**

- **夯实编程基础，为进阶深造打下基础**

课程内容

信息安全基础综合设计实验



➤ **主要资料源于课件**

➤ **参考教材：“信息安全基础综合实验教程”，程红蓉等编著，高等教育出版社，2012.02**

课时安排

主题	课时
Linux编程基础	4
模指数运算、素性测试I	4
素性测试II、乘法逆元	4
线性同余位随机数生成器、BBS伪随机数生成器	4
编程链接库、大数运算	4
RSA加密、RSA签名	4
流密码、分组密码	4
复习	4

➤视情况补充新增的8学时

考核方式

- 平时成绩（**30%**）：课堂实验 + 课后实验
 - 第2-7次课均有实验作业，未能完成，成绩折扣
- 期末成绩（**70%**）：程序+试卷
- 由于平时作业较多，视情况而定**平时-期末成绩比例**和**期末考核方式**

课堂纪律

- 平时作业有20%以上未按时完成
- 实验课缺课学时达到总学时20%以上
- 无正当理由或未经学院同意，缺课学时达到总学时的40%以上

以上情况之一，**取消**考试资格（校教通[2005]106号）

Linux命令行环境

Linux系统配置

➤ 更换国内软件源：阿里云、清华、中科大、163.....

- 更换源后，执行 `sudo apt-get update` 更新软件包索引，以及 `sudo apt-get upgrade` 升级现有软件包
 - `sudo` 为当前命令提升权限，要求输入口令，**口令默认不可见**

➤ 配置编程环境（更多 Linux基础、Vim简单使用、Bash脚本）：

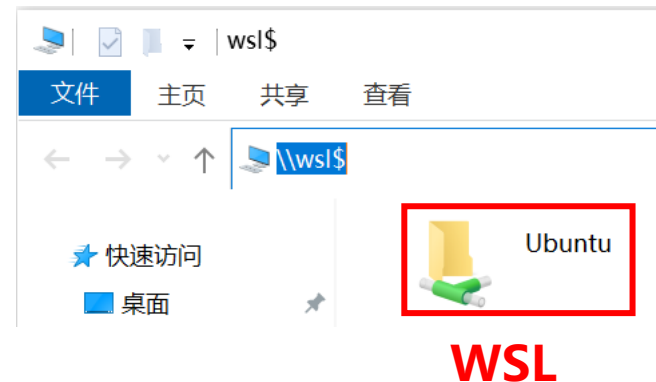
```
# 安装vim和编译工具
$ sudo apt-get install git cmake vim build-essential

# 配置vimrc
$ git clone -depth=1 https://github.com/amix/vimrc.git ~/.vim_runtime
$ sh ~/.vim_runtime/install_basic_vimrc.sh
```

与宿主操作系统互访问

➤ 宿主操作系统访问WSL：`\\wsl$`

- 可通过<win+r>运行窗口或资源管理器地址栏访问
- 用户目录位于 “`\\home\\{用户名}\\`”
 - 可以将作业包解压至WSL的 “`\\home\\{用户名}\\`” 目录

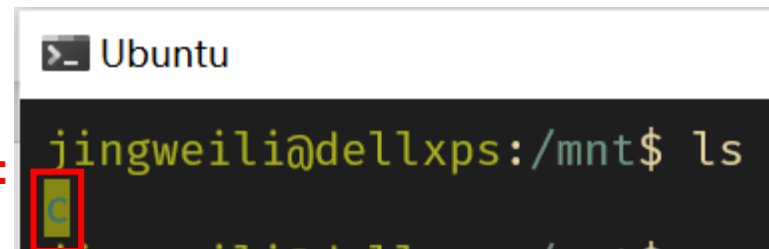


➤ WSL访问宿主操作系统：所有盘符挂载在 “`/mnt`” 目录下

- 勿在WSL中直接修改宿主操作系统文件

➤ Vmware互访问依赖[VM Tools](#)

宿主操作系统盘符
(只有一个C盘)



计算机教育缺失的一课

- 大学计算机课程通常专注于讲授程序设计、数据结构、操作系统、编译原理等**学院派主题**，**但是未涉猎如何精通编程工具**
- 第一次课关注**如何高效使用工具编写程序**

The Missing Semester of Your CS Education

Classes teach you all about advanced topics within CS, from operating systems to machine learning, but there's one critical subject that's rarely covered, and is instead left to students to figure out on their own: proficiency with their tools. We'll teach you how to master the command-line, use a powerful text editor, use fancy features of version control systems, and much more!

Students spend hundreds of hours using these tools over the course of their education (and thousands over their career), so it makes sense to make the experience as fluid and frictionless as possible. Mastering these tools not only enables you to spend less time on figuring out how to bend your tools to your will, but it also lets you solve problems that would previously seem impossibly complex.

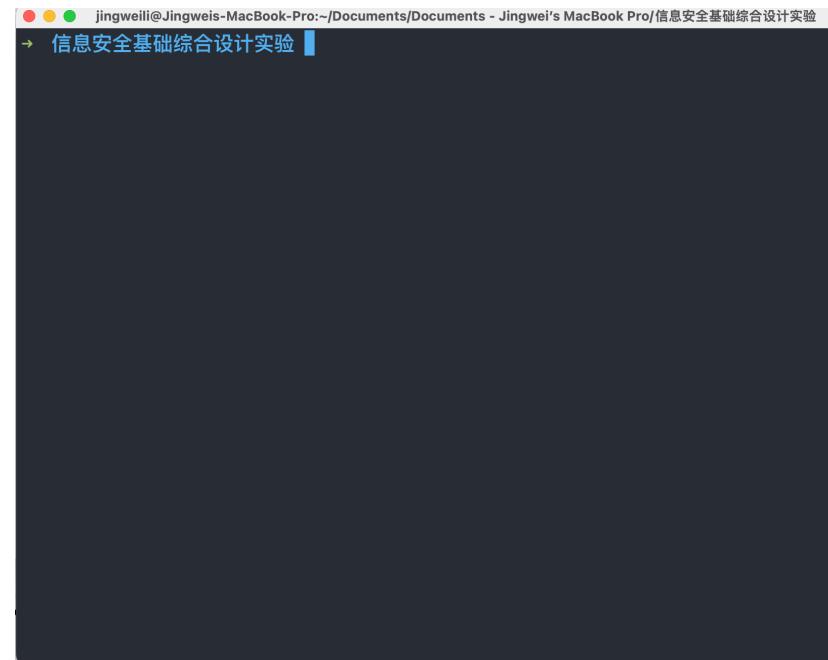
Read about the [motivation behind this class](#).

Schedule

- 1/13/20: [Course overview + the shell](#)
- 1/14/20: [Shell Tools and Scripting](#)
- 1/15/20: [Editors \(Vim\)](#)
- 1/16/20: [Data Wrangling](#)
- 1/21/20: [Command-line Environment](#)
- 1/22/20: [Version Control \(Git\)](#)
- 1/23/20: [Debugging and Profiling](#)

Linux命令行环境

- 在Windows 10/11中，激活WSL环境
- 打开终端（Terminal）进入默认的Shell环境（Ubuntu为Bash）
 - Shell是与操作系统内核交互的一种接口



Shell基础

➤默认提示符：用户名 + 当前工作目录

- &：root用户
- ~：当前用户的主目录

➤无参数指令示例：date

➤带参数指令示例：echo

- 基于空格分割命令并进行解析，如果参数包含空格，使用单引号/双引号包裹参数，或使用转义符号\进行处理

路径

- 所有Shell固有指令来自环境变量PATH
 - 可以通过which指令确定某个程序名代表的是哪个具体的可执行文件
- 路径是一组被 / 分割的目录
 - / 代表系统根目录
 - 绝对路径以 / 开头，相对路径是相对于当前工作目录（可以通过pwd显示）的路径
 - . 表示当前目录，.. 表示上一级目录
- 通过 cd 命令切换当前工作目录

导航

➤通过 ls 显示指定目录下的所有文件

- 未指定目录则显示当前目录下的文件
- 在执行程序时使用 -h 或 --help 标记可以打印帮助信息

➤显示长列表格式 (ls -l)

代表所有者、用户组、其他人的权限

```
> ls -l
-rwxr-xr-x  3 jingweili staff  96 B  Tue Apr  1 23:03:33 2025  ai_completion/
-rwx----- 6 jingweili staff 192 B  Tue Mar 26 14:44:33 2024  Applications/
-rwx-----21 jingweili staff 672 B  Wed Aug 13 09:47:25 2025  Desktop/
-rwxr-xr-x 32 jingweili staff   1 KiB Tue Sep  2 15:36:38 2025  Documents/
-rwx-----20 jingweili staff 640 B  Tue Sep  2 15:21:11 2025  Downloads/
-rwxr-xr-x 59 jingweili staff   1 KiB Tue Dec  7 23:45:33 2021  languageserver-library/
```

其他有用指令

➤ mv 移动文件/目录

➤ cp 拷贝文件

- 拷贝目录须增加 -r 参数

➤ mkdir 创建文件夹

➤ man 帮助指令

重定向

➤ 程序有两个主要的“流”：**输入流**和**输出流**

- 通常情况下，键盘是输入流，终端是输出流

➤ `< file` 和 `> file` 将程序的输入和输出重定向到文件

```
$ echo hello > hello.txt
$ cat hello.txt
hello
$ cat < hello.txt
hello
$ cat < hello.txt > hello2.txt
$ cat hello2.txt
hello
```

Hello World

➤ Hello World程序

- 保存为文件hello.c

➤ 编译：g++ -o hello hello.c

- -o：指定产生可执行文件文件名

➤ 运行：./hello

```
#include <stdio.h>
int main () {
    printf("hello
world\n");
    return 0;
}
```

Makefile简介

➤Linux程序编译复杂

- 耗力：每个小模块都需要指令来编译
- 耗时：修改任意部分导致整个程序重新编译

➤**Makefile**由一系列规则构成，指明编译顺序

- 省力：**make**编译产生可执行文件
- 省时：通过源文件修改时间来判断哪些规则需要重新编译

Makefile规则

➤ **TARGET**：规则生成目标文件

- 通常是需要生成的程序名或过程文件

```
TARGET: PREREQUISITE ...  
        COMMAND ...  
        COMMAND ...
```

➤ **PREREQUISITE**：规则依赖项

- 通常是依赖的源程序文件

➤ **COMMAND**：规则所需执行的命令行

- 通常是编译命令，**每一行命令以[TAB]字符开头**
- 只有**PREREQUISITE更新时间晚于TARGET**，COMMAND才能执行

Hello World项目

➤假设项目有三个文件

```
// sayHello.h  
void sayHello();
```

```
// sayHello.c  
#include <stdio.h>  
#include <sayHello.h>  
  
void sayHello () {  
    printf("hello world\n");  
}
```

```
// main.c  
#include <sayHello.h>  
  
int main () {  
    sayHello();  
    return 0;  
}
```

命令行直接编译

➤编译命令：`g++ -o hello main.c sayHello.c -I.`

- `-I.`将当前目录纳入头文件搜索路径

➤问题：

- 每次都需要重新输入命令
- 如果只改变其中一个C文件，命令行重新编译须编译所有文件

Makefile I

```
# 命名为makefile
```

```
helloWorld: sayHello.c main.c  
    g++ -o helloWorld sayHello.c main.c -I.
```

➤通过**make**命令执行

- 当**make**不指定参数时，默认读取makefile的第一个规则

Makefile II

```
CC=g++  
CFLAGS=-I.  
  
helloWorld: sayHello.o main.o  
    $(CC) -o helloWorld sayHello.o main.o $(CFLAGS)
```

- 定义**常量**，表示编译器和编译选项
 - 方便修改
- Make命令读取.o依赖时，自动将相应源文件编译为目标.o文件

Makefile III

➤ 头文件纳入规则

- 头文件变化会重新编译

➤ 引入通配符：%匹配零或若干字符

➤ 引入变量

- **常用自动变量**

```
CC=g++
CFLAGS=-I.
DEPS=sayHello.h

helloWorld: main.o sayHello.o
    $(CC) -o helloWorld $^ $(CFLAGS)

%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)
```

\$@	指代规则目标
\$<	指代规则的第一项依赖
\$^	指代规则的所有依赖

Makefile IV

```
CC=g++
CFLAGS=-I.
DEPS=sayHello.h
OBJ=main.o sayHello.o

helloWorld: $(OBJ)
    $(CC) -o helloWorld $^ $(CFLAGS)

%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)
```

➤使用常量OBJ指代所有目标文件

Makefile V (完整版)

- 支持层次组织结构：头文件→`./include`；源文件→`./src`；
本地库文件→`./lib`；目标文件→`./src/obj`
- 支持定义引入的编程库
- 支持清理可执行文件和目标文件

```
CC=g++
TARGET=helloWorld
```

假设makefile位于目录src/内

```
ODIR=./obj
IDIR=../include
LDIR=../lib
```

目录结构

```
CFLAGS=-I$(IDIR)
LIBS=
```

编译参数（头文件位置和链接库名称）

```
_DEPS=sayHello.h
DEPS=$(patsubst %, $(IDIR)/%, $_DEPS)
_OBJ=main.o sayHello.o
OBJ=$(patsubst %, $(ODIR)/%, $_OBJ)
```

生成完整头文件路径和目标文件路径：

`$(patsubst pattern, replacement, text)`

- 匹配`text`中的单词（以空格分隔）与`pattern`，如果匹配成功，则以`replacement`替换
- `pattern`可以包含通配符%，如果同时`replacement`也包含%，那么`replacement`的%和`pattern`的%指代相同字串

```
$(ODIR)/%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)

$(TARGET): $(OBJ)
    $(CC) -o $@ $^ $(CFLAGS) $(LIBS)
```

```
clean:
    rm -f $(ODIR)/*.o $(TARGET)
```

删除产生的临时文件和可执行文件

```
.PHONY: clean
```

强制执行clean规则

课后作业

进制转化

➤实现二进制与十进制转换函数

- 二进制数以字符串 (std::string) 表示 ; 十进制数以无符号整数 (unsigned int) 表示

➤函数体 :

```
unsigned int binary_to_decimal (std::string input);  
  
std::string decimal_to_binary (unsigned int input);
```